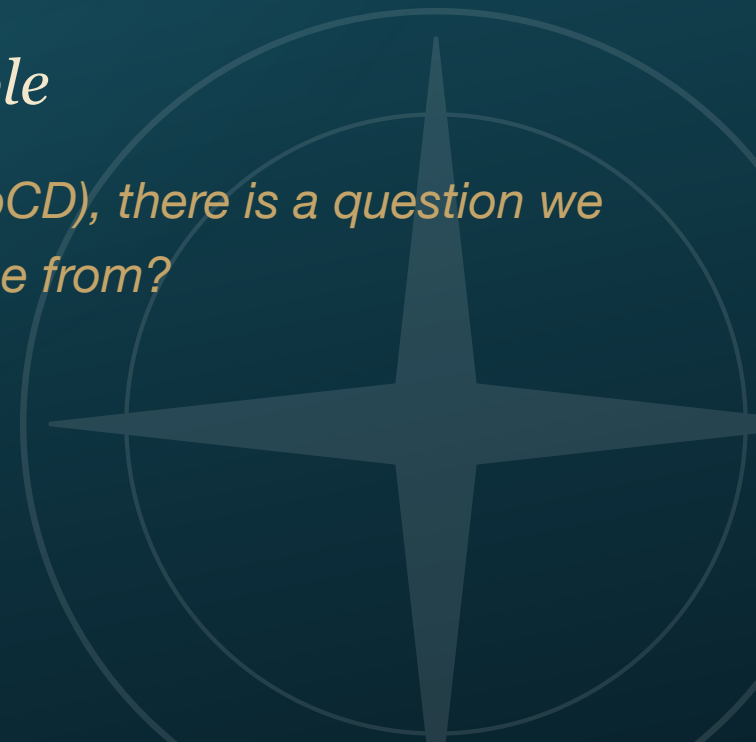


DAY 3 · OPENING DEMO · 30 MINUTES

Gitea Actions

In-cluster CI, GitHub-Actions-compatible

Before we template the manifests (Helm) and before we deploy them (ArgoCD), there is a question we have been quietly skipping: where does the image come from?



Where does the image come from?

- ♦ For two days, we built the images — locally, with `docker build`, then `docker push` to Harbor.
- ♦ That isn't a workflow. That's the instructor cheating off-stage.
- ♦ After lunch you'll meet **ArgoCD**, which pulls images from Harbor and ships them to the cluster. But ArgoCD assumes the image is **already there**.
- ♦ This morning we fix that: a `git push` automatically produces a tagged image in Harbor, with no human in the middle.

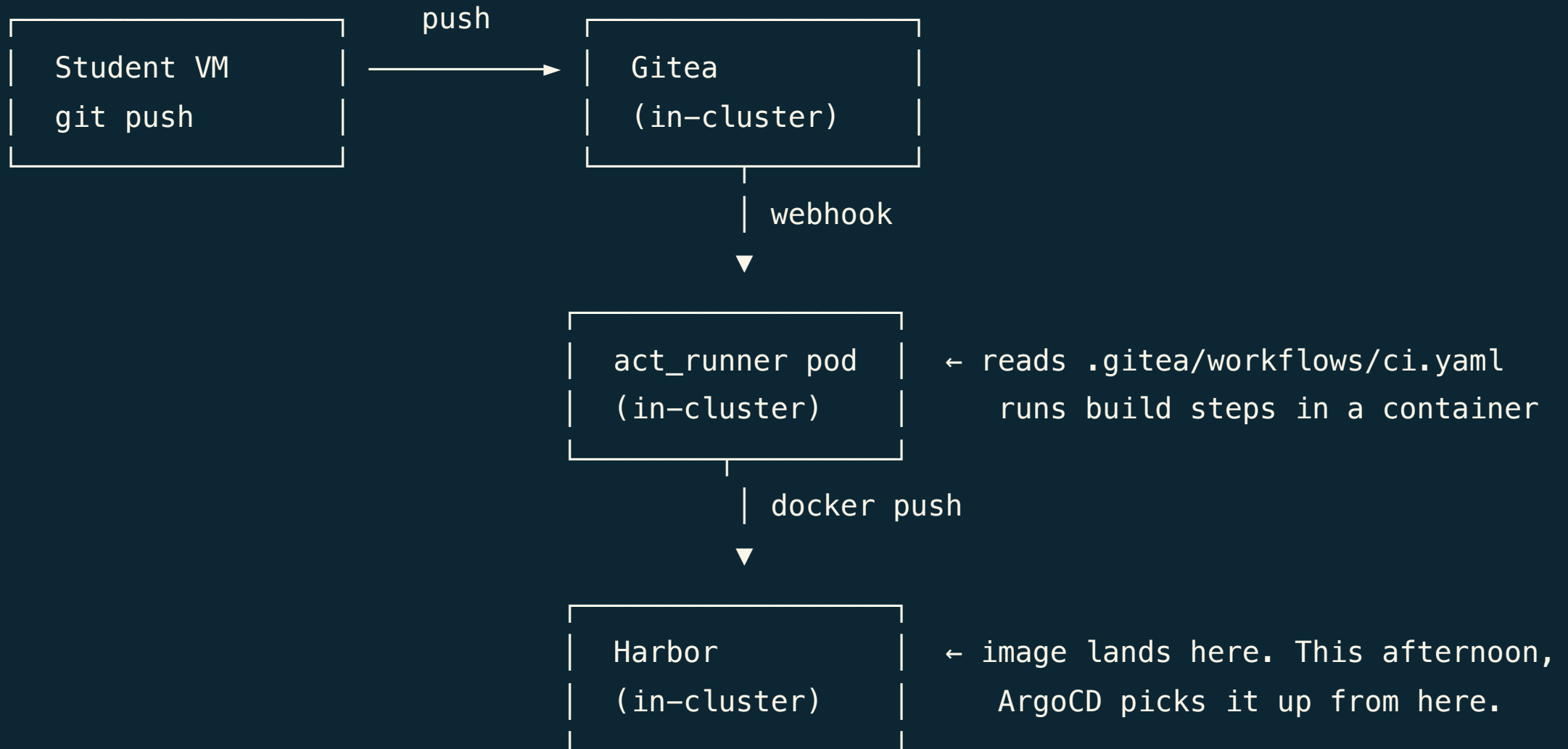
Today we build the build half. After lunch ArgoCD finishes the loop.

The committed shape (and what we actually built)

- ♦ The original course outline named **GitHub Actions**: workflows, jobs, secrets, build-and-push.
- ♦ We will **not** use GitHub Actions in the cluster. The room has no public internet path and the cluster shouldn't depend on github.com.
- ♦ Instead: **Gitea Actions** — the same workflow YAML, executed by an in-cluster runner against our in-cluster Gitea.
- ♦ *Same mental model. Different home.*

Concept	GitHub Actions	Gitea Actions
Workflow file	<code>.github/workflows/ci.yaml</code>	<code>.gitea/workflows/ci.yaml</code>
Runner	GitHub-hosted or self-hosted	<code>act_runner</code> pod in our cluster
Image cache	github.com Actions cache	local registry / runner volume
Marketplace	<code>uses: docker/build-push-action@v5</code>	same — Gitea Actions is <code>act</code> -compatible

What's actually running



The workflow file

```
# .gitea/workflows/ci.yaml
name: build-and-push
on:
  push: { branches: [main] }
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Log in to Harbor
        uses: docker/login-action@v3
        with:
          registry: harbor.${{ vars.LAB_DOMAIN }}
          username: ${ secrets.HARBOR_ROBOT_USER }
          password: ${ secrets.HARBOR_ROBOT_SECRET }
      - name: Build and push
        uses: docker/build-push-action@v6
        with:
          push: true
          tags: harbor.${{ vars.LAB_DOMAIN }}/raft-fleet/raft:${{ github.sha }}
```

That's it. Same syntax a working Actions engineer writes on day one.

Live demo — push to image

We're going to push a one-character change to a repo and watch it produce a tagged image in Harbor, with no human in the middle.

1. **Edit** a string in the `raft` repo's `index.html`.
2. **Commit + push** to Gitea.
3. Watch the **Actions** tab in Gitea — runner picks up the job within 5 seconds.
4. Watch the build logs stream in real time.
5. Watch **Harbor** — a new tag appears under `raft-fleet/raft` when push completes.

From `git push` to tagged image in under 60 seconds. The image is now sitting in Harbor, waiting for something to deploy it.

WHAT'S NEXT ON THE SHELF

Beyond Gitea Actions

~~~~^~~~~~^~~~~~^~~~~~^~~~~~^~~~~~^~~~~~^~~~~~  
~~~~~^~~~~~^~~~~~^~~~~~^~~~~~^~~~~~^~~~~~^~~~~~^~~~~~  
~^~~~~~^~~~~~^~~~~~^~~~~~^~~~~~^~~~~~^~~~~~^~~~~~^~~~~~



When Gitea Actions stops being enough

- ♦ **Gitea Actions** is the right answer for: classroom labs, small teams, GitHub-compatible workflows.
- ♦ It's the *wrong* answer when you need:
 - DAG topologies (fan-out / fan-in across many steps with conditional dependencies)
 - **CRD-native** pipelines that other Kubernetes tools can introspect
 - First-class CNCF graduation status for your platform's compliance review

Two CNCF-graduated projects exist for exactly that.

Argo Workflows — the Argo family answer

- ♦ **CNCF graduated.** Same project family as ArgoCD.
- ♦ Pipelines are CRDs — `Workflow`, `WorkflowTemplate`, `CronWorkflow`.
- ♦ Designed around **DAGs** — every step is a container, dependencies are explicit.
- ♦ Argo **Events** (sibling project) gives you webhook / git / S3 / kafka triggers.
- ♦ Most natural fit if you're already standing up ArgoCD — same UI conventions, same RBAC story.

```
apiVersion: argoproj.io/v1alpha1
kind: Workflow
spec:
  templates:
    - name: build
      container: { image: gcr.io/kaniko-project/executor, args: [...]
}
```

Tekton — the industry-standard answer

- ♦ **CNCF graduated.** The reference Kubernetes-native CI engine.
- ♦ Pipelines are CRDs — `Task`, `Pipeline`, `PipelineRun`, `TaskRun`.
- ♦ 1:1 conceptual map to GitHub Actions: workflow → pipeline, job → task, step → step.
- ♦ **Tekton Hub** is a large catalog of reusable Tasks (build, scan, sign, deploy).
- ♦ The one you'll see in industry CI/CD platforms (OpenShift Pipelines is Tekton, Jenkins X uses Tekton, etc.).

```
apiVersion: tekton.dev/v1
kind: Pipeline
spec:
  tasks:
    - name: build-image
      taskRef: { name: buildah, kind: ClusterTask }
```

How to choose, in one sentence

| If you want... | Reach for... |
|---|-----------------------|
| GitHub Actions semantics, in-cluster, today | Gitea Actions |
| DAGs in the same UI as your ArgoCD | Argo Workflows |
| CNCF reference platform, big task catalog | Tekton |

All three run on the same cluster you already have. None of them is wrong.

Now: meet Hazel.

You've seen the build half. The template + deploy halves are the rest of today.

